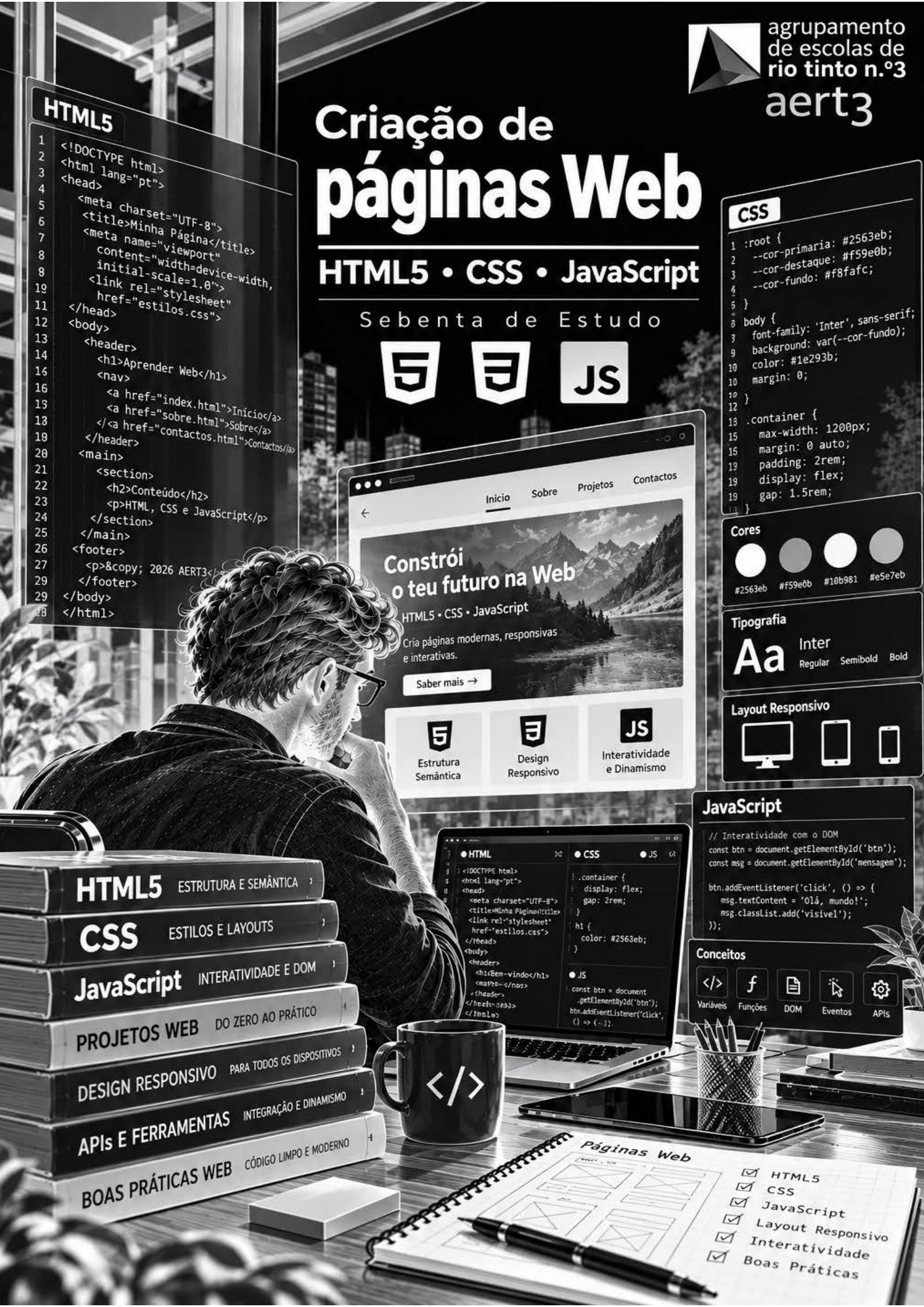

agrupamento de escolas de rio tinto n.º3
aert3
Criação de páginas Web
HTML5 • CSS • JavaScript
Sebenta de Estudo
JS
HTML5
<!DOCTYPE html>
<html lang="pt">
<head>
<meta charset="UTF-8">
<title>Minha Página</title>
<meta name="viewport"
content="width=device-width,
initial-scale=1.0">
<link rel="stylesheet"
href="estilos.css">
</head>
<body>
<header>
<h1>Aprender Web</h1>
<nav>
<a href="index.html">Início</a>
<a href="sobre.html">Sobre</a>
<a href="contactos.html">Contactos</a>
</header>
<main>
<section>
<h2>Conteúdo</h2>
HTML, CSS e JavaScript
</section>
</main>
<footer>
© 2026 AERT3
</footer>
</body>
</html>
CSS
:root {
--cor-primaria: #2563eb;
--cor-destaque: #f59e0b;
--cor-fundo: #f8fafc;
}
body {
font-family: 'Inter', sans-serif;
background: var(--cor-fundo);
color: #1e293b;
margin: 0;
}
.container {
max-width: 1200px;
margin: 0 auto;
padding: 2rem;
display: flex;
gap: 1.5rem;
}
Inicio
Sobre
Projetos
Contactos
Constrói o teu futuro na Web
HTML5 • CSS • JavaScript
Cria páginas modernas, responsivas e interativas.
Saber mais →
Estrutura Semântica
Design Responsivo
Interatividade e Dinamismo
Cores
#2563eb
#f59e0b
#10b981
#e5e7eb
Tipografia
Aa
Inter
Regular
Semibold
Bold
Layout Responsivo
JavaScript
// Interatividade com o DOM
const btn = document.getElementById('btn');
const msg = document.getElementById('mensagem');
btn.addEventListener('click', () => {
msg.textContent = 'Olá, mundo!';
msg.classList.add('visivel');
});
Conceitos
Variáveis
Funções
DOM
Eventos
APIs
HTML
CSS
JS
HTML5 ESTRUTURA E SEMÂNTICA
CSS ESTILOS E LAYOUTS
JavaScript INTERATIVIDADE E DOM
PROJETOS WEB DO ZERO AO PRÁTICO
DESIGN RESPONSIVO PARA TODOS OS DISPOSITIVOS
APIs E FERRAMENTAS INTEGRAÇÃO E DINAMISMO
BOAS PRÁTICAS WEB CÓDIGO LIMPO E MODERNO
Páginas Web
HTML5
CSS
JavaScript
Layout Responsivo
Interatividade
Boas Práticas

# Criação de Páginas Web - *Aprender fazendo*

Do primeiro **Olá Mundo!** a um website responsivo com três páginas e interatividade.

| HTML | CSS | JavaScript |
|---|---|---|
| Estrutura e conteúdo | Aspeto e disposição | Comportamento e interação |
| Títulos, imagens, formulários | Cores, espaço, Flexbox, Grid | Eventos, DOM e respostas ao utilizador |

## Para quem é esta sebenta?

Alunos do ensino profissional que estão a iniciar o desenvolvimento Web. Não é necessário saber programar. É suficiente conseguir criar pastas, guardar ficheiros e utilizar um navegador.

## O projeto que te acompanha

Vamos construir o **Clube Web**, um pequeno website sobre atividades de uma escola. Primeiro existe apenas uma página. Depois acrescentamos conteúdos, páginas, estilos e comportamentos. Podes adaptar o tema aos teus interesses.

**COMO TRABALHAR**
Lê a explicação, escreve o código, observa o resultado e realiza a atividade. Altera uma coisa de cada vez. Quando houver um erro, compara o resultado com o que esperavas e procura a causa.

**Material necessário:** VS Code (por exemplo, 1.138.0), navegador e uma pasta de trabalho. O Live Server é útil para pré-visualizar; não é obrigatório para os primeiros exercícios.

**Convenção:** os exemplos HTML são colocados dentro de <body>, salvo indicação em contrário. O CSS vai para css/estilo.css e o JavaScript para js/script.js. Os nomes dos menus do editor podem variar com o idioma.

**FICHEIROS DE APOIO**
O ZIP que acompanha a sebenta contém três versões do Clube Web: HTML, HTML com CSS e versão final com JavaScript. Extrai o ZIP antes de abrir uma das pastas no VS Code.

# Índice e percurso


| Capítulo | Páginas |
|---|---|
| 01 • Introdução | 3 |
| 02 • HTML5 | 4-5 |
| 03 • Elementos HTML | 6-12 |
| 04 • Tabelas | 13-14 |
| 05 • Formulários | 15-17 |
| 06 • HTML semântico | 18-19 |
| 07 • Multimédia | 20 |
| 08 • iframe | 21 |
| 09 • Introdução ao CSS | 22-23 |
| 10 • Seletores CSS | 24-25 |
| 11 • Propriedades CSS | 26-28 |
| 12 • Box Model | 29 |
| 13 • Flexbox | 30-31 |
| 14 • CSS Grid | 32 |
| 15 • Responsividade | 33-35 |
| 16 • JavaScript | 36-38 |
| 17 • JavaScript e DOM | 39-42 |
| 18 • Projeto progressivo | 43-44 |
| 19 • Prática autónoma | 45 |
| 20 • Boas práticas | 46 |
| 21 • Projeto final | 47-48 |
| 22 • Soluções comentadas | 49-50 |
| 23 • Consulta e recursos | 51 |


**Percurso:** HTML (01-08),
CSS e responsividade (09-15),
JavaScript (16-17),
consolidação e projeto (18-23).
As tabelas no final dos capítulos servem de consulta rápida.

**Caixas de aprendizagem:** Dica / Boa prática, Atenção / Erro frequente, Experimenta tu e Desafio. Os exercícios curtos duram cerca de 5 a 15 minutos; os mini projetos podem ocupar várias aulas.

# Da página ao website

Uma **página Web** é um documento que o navegador apresenta. Um **website** é um conjunto organizado de páginas e recursos relacionados. O ficheiro sobre.html pode ser uma página do website da escola.

O **navegador**, como Firefox, Edge ou Chrome, obtém os ficheiros, interpreta HTML e CSS e executa JavaScript. O VS Code é o editor onde escreves esses ficheiros; não é o navegador.

| Analogia: uma escola | Na Web | Exemplo |
|---|---|---|
| Salas e conteúdos | HTML organiza a informação | Título e horário |
| Pintura e disposição | CSS apresenta a informação | Cores e grelha |
| Campainha e automatismos | JavaScript reage a ações | Botão que mostra um aviso |

Um website pode ser **estático**: o servidor entrega ficheiros já preparados. Ter JavaScript não implica ter uma base de dados. Um formulário real precisa também de um serviço que receba os dados.

## Criar a pasta de projeto

Cria meu_site nos Documentos. No VS Code, usa **File > Open Folder**. Dentro da pasta, cria esta estrutura:

```
meu_site/
  index.html
  css/
    estilo.css
  js/
    script.js
  imagens/
```

**BOA PRÁTICA**
Usa nomes curtos, em minúsculas e sem espaços: sobre.html, foto-escola.jpg. Confirma que o ficheiro não ficou com o nome index.html.txt.

**EXPERIMENTA TU**
Cria as pastas e os três ficheiros. Explica a um colega a diferença entre uma página e um website. Resultado: uma pasta organizada, mesmo que os ficheiros ainda estejam vazios.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| Página | Um documento | sobre.html |
| Website | Páginas e recursos | Clube Web |
| Navegador | Interpreta a página | Firefox / Edge |

# A primeira página completa

HTML é uma linguagem de marcação. Indica o significado das partes do conteúdo. Escreve este documento completo em **index.html**:

```
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">
    <title>A minha primeira página</title>
</head>
<body>
    <h1>A minha primeira página</h1>
    <p>Olá Mundo!</p>
</body>
</html>
```

| Linha / elemento | O que significa |
|---|---|
| <!DOCTYPE html> | Declara que se utiliza HTML moderno; não é uma tag de conteúdo. |
| <html lang="pt"> | Inicia o documento e indica que o conteúdo está em português. |
| <head> | Abre a área de informação sobre o documento. |
| <meta charset="UTF-8"> | Define a codificação, permitindo acentos e outros caracteres. |
| meta viewport | Adapta a área de visualização à largura do dispositivo. |
| <title> | Define o título do separador do navegador. |
| </head> e <body> | Fecha a informação do documento e abre o conteúdo apresentado. |
| <h1> e <p> | Criam o título principal e um parágrafo. |
| </body> e </html> | Fecham o corpo e o documento. |

**Resultado:** vês um título e “Olá Mundo!” na página. O separador mostra “A minha primeira página”. Guarda com **Ctrl + S** e abre o ficheiro no navegador ou usa **Open with Live Server**.

**EXPERIMENTA TU**
Altera o título do separador para Clube Web e o h1 para Bem-vindo ao Clube Web.
Guarda e compara: os dois títulos surgem em locais diferentes.

# Ler e escrever marcação

```
<p class="aviso">Inscrições abertas!</p>
```

| Conceito | Neste exemplo |
|---|---|
| Tag de abertura |  inicia o parágrafo. |
| Tag de fecho |  termina o parágrafo. |
| Elemento | Todo o conjunto: abertura, conteúdo e fecho. |
| Atributo | class="aviso" acrescenta informação ao elemento. |
| Conteúdo | Inscrições abertas! é o texto entre as tags. |

Os elementos **vazios**, como img, br e meta, não têm conteúdo nem tag de fecho. Escreve <img ...>, não </img>. Os elementos devem ser encaixados corretamente: o último que abres é o primeiro que fechas.

```
<!-- Este comentário explica a secção seguinte. -->
<p>O clube é <strong>gratuito</strong>.</p>
```

A **indentação** é o avanço à esquerda que ajuda a perceber o encaixe. Não cria margens visuais na página. Um comentário não aparece como conteúdo, mas pode ser lido no código: nunca coloques palavras-passe em comentários.

**ERRO FREQUENTE**
Usa as tags de fecho nos exemplos. Em HTML, alguns fechos, como o de p em certos contextos, podem ser omitidos; depender disso torna o código mais difícil de ler e corrigir.

**ENCONTRA O ERRO • 5 MINUTOS**
Corrige: Olá turma! . Primeiro identifica qual foi o último elemento aberto. Solução comentada no final.

**EXPERIMENTA TU**
Acrescenta dois parágrafos ao Clube Web, um deles com uma palavra importante. Insere um comentário útil antes desses parágrafos.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| Tag | Marca o início / fim |  /  |
| Atributo | Informação adicional | lang="pt" |
| Vazio | Sem tag de fecho | <br> |
| Comentário | Nota no código | <!-- Menu --> |

# Estrutura: o que fica onde?

## html | Raiz do documento

**Sintaxe:** `<html lang="pt">...</html>`

```
<html lang="pt"> ... </html>
```

**Resultado:** Envolve o documento; não desenha uma caixa por si só. **Dica:** Usa uma única raiz html.

## head | Informação sobre a página

**Sintaxe:** `<head>...</head>`

```
<head><title>Clube Web</title></head>
```

**Resultado:** O título aparece no separador; o head não é a área de conteúdo. **Dica:** Não coloques aqui os parágrafos da página.

## meta | Metadados; elemento vazio

**Sintaxe:** `<meta atributo="valor">`

```
<meta charset="UTF-8">
```

**Resultado:** Os caracteres são interpretados com a codificação indicada.

## title | Nome do documento

**Sintaxe:** `<title>...</title>`

```
<title>Atividades | Clube Web</title>
```

**Resultado:** O separador identifica a página de atividades.

## body | Conteúdo do documento

**Sintaxe:** `<body>...</body>`

```
<body><h1>Clube Web</h1></body>
```

**Resultado:** O título aparece dentro da página. **Dica:** Usa um único body.

# Títulos, parágrafos e separadores

## h1 a h6 | Hierarquia de títulos

**Sintaxe:** `<hN>...</hN> (N de 1 a 6)`

```
<h1>Clube Web</h1>
<h2>Atividades</h2>
<h3>Oficina de HTML</h3>
```

**Resultado:** Três níveis de títulos. h1 é o nível principal; h6 é o sexto nível. **Dica:** Escolhe o nível pelo significado, não pelo tamanho. Neste projeto usa um h1 por página.

Os níveis seguintes usam a mesma sintaxe: <h4>Materiais</h4>, <h5>Recursos digitais</h5> e <h6>Notas do recurso</h6>. Só precisas deles quando a hierarquia do conteúdo chega a esses níveis.

## p | Parágrafo

**Sintaxe:** `<p>...</p>`

```
<p>Aprendemos a criar páginas Web.</p>
```

**Resultado:** Um bloco de texto, normalmente separado por espaço dos restantes.

## br | Quebra de linha; elemento vazio

**Sintaxe:** `<br>`

```
<p>Sala 12<br>Edifício principal</p>
```

**Resultado:** Sala e edifício ficam em linhas diferentes. **Dica:** Não repitas br para criar margens; usa CSS.

## hr | Mudança temática; elemento vazio

**Sintaxe:** `<hr>`

```
<p>Notícias</p><hr><p>Arquivo</p>
```

**Resultado:** Separa dois temas; normalmente surge uma linha horizontal.

**EXPERIMENTA TU**
Cria duas secções de texto, cada uma com h2 e p. Acrescenta uma morada em duas linhas. Resultado: consegues identificar a hierarquia apenas pelos títulos.

# Dar significado ao texto

| Elemento e função | Sintaxe / exemplo | Resultado habitual e dica |
|---|---|---|
| strong: importância | Atenção | Negrito; o significado é importância. |
| em: ênfase | mesmo | Itálico; marca ênfase na leitura. |
| mark: relevância | <mark>HTML</mark> | Realce, normalmente amarelo. |
| small: nota secundária | <small>Uso educativo</small> | Texto menor, como uma nota legal. |
| sub: índice inferior | H<sub>2</sub>O | O 2 fica abaixo da linha: $H_2O$. |
| sup: índice superior | m<sup>2</sup> | O 2 fica acima da linha: $m^2$. |

A sintaxe de cada elemento é uma tag de abertura, o conteúdo e a tag de fecho. A apresentação habitual pode ser alterada por CSS: não escolhas strong apenas por quereres negrito.

```
<p><strong>Inscreve-te</strong> até sexta-feira.</p>
<p>Esta oficina é <em>mesmo</em> para principiantes.</p>
<p>Pesquisa: <mark>HTML</mark></p>
<p><small>Atividade sujeita a lotação.</small></p>
<p>Água: H<sub>2</sub>O. Área: 20 m<sup>2</sup>.</p>
```

**EXPERIMENTA TU**
Acrescenta um aviso com strong, uma expressão com em e uma nota com small. Explica por que escolheste cada elemento.

**QUAL SERÁ O RESULTADO? • 5 MINUTOS**
Em 2<sup>3</sup> , o navegador calcula 8 ou apresenta o expoente? Justifica antes de testar.

## Organizar sem acrescentar significado

**div** é um contentor genérico, normalmente apresentado em bloco. **span** é um contentor genérico em linha, útil para uma pequena parte de texto. Prefere um elemento semântico quando existir um adequado.

```
<div class="cartao">
    <p>Estado: <span class="destaque">aberto</span></p>
</div>
```

**Sintaxe:** … e …. **Resultado:** uma área com um parágrafo; o span não tem aspeto especial sem CSS.

# Listas e hiperligações

| Elemento e função | Sintaxe | Exemplo e resultado |
|---|---|---|
| ul: lista sem ordem | <ul><li>...</li></ul> | <ul><li>HTML</li></ul>: item com marcador. |
| ol: lista ordenada | <ol><li>...</li></ol> | <ol><li>Abrir</li></ol>: item numerado. |
| li: item de uma lista | <li>...</li> | <li>Guardar</li>: um item dentro de ul ou ol. |

```
<h2>O que vamos aprender</h2>
<ul>
    <li>HTML</li>
    <li>CSS</li>
</ul>
<h2>Como começar</h2>
<ol>
    <li>Abrir o editor</li>
    <li>Criar o ficheiro</li>
</ol>
```

**Resultado:** a primeira lista tem marcadores; a segunda mostra uma sequência. **Dica:** coloca os itens dentro de li, não diretamente em ul.

## a | Hiperligação

**Sintaxe:** `<a href="destino">Texto</a>`

```
<a href="sobre.html">Sobre o clube</a>
```

**Resultado:** Ao ativar a ligação, o navegador abre sobre.html na mesma pasta.

| Atributo | Função | Exemplo |
|---|---|---|
| href | Destino da ligação | href="sobre.html" |
| target | Onde abrir | target="_blank" |
| title | Informação complementar | title="Conhece a equipa" |

**DICA**
O texto da ligação deve explicar o destino. “Horário das oficinas” é mais útil do que “clica aqui”. O atributo title não substitui um texto claro e pode não ser acessível em dispositivos táteis.

# Ligar páginas e outros destinos

```
<!-- Outra página do mesmo website -->
<a href="sobre.html">Sobre o clube</a>

<!-- Website externo -->
<a href="https://developer.mozilla.org/">Aprender na MDN</a>

<!-- Abre a aplicação de correio configurada -->
<a href="mailto:clube@example.com">Escrever ao clube</a>

<!-- Novo separador -->
<a href="https://developer.mozilla.org/"
   target="_blank" rel="noopener noreferrer">
    MDN (abre num novo separador)
</a>
```

**Resultado:** cada ligação abre o destino indicado. mailto não envia uma mensagem automaticamente: depende da aplicação de correio configurada. O endereço example.com é ilustrativo.

rel="noopener noreferrer" evita disponibilizar a página de origem como opener e omite o cabeçalho de referência. Ao abrir um novo separador, informa o utilizador no texto.

## Também podes ligar a uma zona da página

```
<a href="#horario">Ver horário</a>
<h2 id="horario">Horário das oficinas</h2>
```

**Resultado:** a ligação desloca a página até ao elemento com id="horario". O identificador deve ser único nesse documento.

**EXPERIMENTA TU**
Cria sobre.html com a estrutura completa. Na página inicial, liga a sobre.html; nessa página, cria uma ligação de regresso a index.html. Testa as duas direções.

**ERRO FREQUENTE**
Se escreves sobre.html mas o ficheiro se chama Sobre.html, a ligação pode falhar ao publicar. Mantém exatamente as mesmas letras e extensões.

# Imagens e caminhos relativos

## img | Imagem; elemento vazio

**Sintaxe:** `<img src="caminho" alt="descrição">`

```
<img src="imagens/escola.jpg"
     alt="Entrada principal da escola"
     width="800" height="450">
```

**Resultado:** Uma imagem com proporção 800 × 450. O ficheiro tem de existir. **Dica:** Usa as dimensões reais; depois adapta o tamanho com CSS.

| Atributo | Função | Exemplo |
|---|---|---|
| src | Caminho do ficheiro | src="imagens/escola.jpg" |
| alt | Alternativa textual | alt="Entrada principal da escola" |
| width / height | Dimensões em píxeis; ajudam a reservar espaço | width="800" height="450" |

Um **caminho relativo** parte da pasta do ficheiro HTML atual. A sequência ../ significa “subir uma pasta”.

```
meu_site/
  index.html
  imagens/escola.jpg
  paginas/sobre.html
```

```
<!-- Em index.html: entra diretamente em imagens -->
<img src="imagens/escola.jpg" alt="Entrada da escola">

<!-- Em paginas/sobre.html: sobe, depois entra em imagens -->
<img src="../imagens/escola.jpg" alt="Entrada da escola">
```

**ERRO FREQUENTE**
Não uses D:\Fotos\escola.jpg nem file:///...: esse caminho só faz sentido no teu computador. Nos caminhos Web usa /, não a barra invertida do Windows.

**EXPERIMENTA TU**
Coloca uma imagem tua em imagens. Mostra-a na página inicial. Para uma imagem decorativa, usa alt=""; para conteúdo importante, descreve a informação que a imagem transmite.

# Consolidar HTML básico

## Mini projeto 1 | Página pessoal

**Objetivo:** Apresentar-te com HTML, sem depender de CSS.

**Ficheiros:** index.html e imagens/retrato.jpg (ou outra imagem autorizada).

**Requisitos obrigatórios:** Um h1, dois h2, três parágrafos, uma lista, uma imagem com alt e uma ligação externa.

**Etapas:** 1. Escreve a estrutura completa. 2. Organiza os conteúdos. 3. Acrescenta a imagem. 4. Testa todas as ligações.

**Resultado esperado:** Uma página legível, com hierarquia e caminhos válidos.

**DESAFIO**
Acrescenta uma ligação para uma secção da própria página e explica o papel do id.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| h1 a h6 | Níveis de títulos | <h2>Atividades</h2> |
| p / br / hr | Texto e separação | Sala 1<br>Piso 2 |
| strong / em | Importância / ênfase | Prazo |
| div / span | Contentores genéricos | aberto |
| ul / ol / li | Listas e itens | <ul><li>HTML</li></ul> |
| a | Hiperligação | <a href="sobre.html">Sobre</a> |
| img | Imagem | <img src="foto.jpg" alt="..."> |

**COMPLETA O CÓDIGO • 5 MINUTOS**
Completa <a ____="sobre.html">Sobre</a> e <img src="foto.jpg" ____="Retrato de Ana">. Não alteres os valores fornecidos.

**CORRIGE O CÓDIGO • 10 MINUTOS**
Em paginas/sobre.html aparece <img src="imagens/escola.jpg">. A pasta imagens está na raiz. Corrige o caminho e acrescenta uma alternativa textual.

# Representar dados em linhas e colunas

Usa tabelas para dados relacionados, como um horário. Não as uses para posicionar o menu e o rodapé. Cada linha contém células, e os cabeçalhos explicam o significado dos dados.

| Elemento | Função e sintaxe | Exemplo / resultado |
|---|---|---|
| table | Envolve a tabela: <table>…</table> | Uma tabela completa. |
| caption | Título: <caption>…</caption> | <caption>Oficinas</caption>: nome da tabela. |
| thead | Agrupa cabeçalhos: <thead>…</thead> | Reúne as linhas de cabeçalho. |
| tbody | Agrupa dados: <tbody>…</tbody> | Reúne as oficinas e durações. |
| tfoot | Agrupa resumo: <tfoot>…</tfoot> | Reúne a linha do total. |
| tr | Linha: <tr>…</tr> | <tr><td>HTML</td></tr>: uma linha. |
| th | Cabeçalho: <th scope="col">…</th> | <th scope="col">Oficina</th>: título de coluna. |
| td | Dado: <td>…</td> | <td>HTML</td>: uma célula de dados. |

## Exemplo de resultado

| Oficina | Duração |
|---|---|
| HTML | 2 h |
| CSS | 3 h |
| Total | 5 h |

O HTML da página seguinte produz estes dados. As cores e linhas desta sebenta são apresentação editorial: o navegador não desenha necessariamente bordas sem CSS.

**BOA PRÁTICA**
Usa caption para identificar a tabela e scope="col" nos cabeçalhos de coluna. Os leitores de ecrã beneficiam destas relações explícitas.

**EXPERIMENTA TU**
Antes de escrever código, desenha uma tabela de três oficinas. Identifica o título, os cabeçalhos e os dados. Depois adapta o exemplo seguinte.

# Construir a tabela do clube

```
<table>
    <caption>Duração das oficinas</caption>
    <thead>
        <tr>
            <th scope="col">Oficina</th>
            <th scope="col">Duração</th>
        </tr>
    </thead>
    <tbody>
        <tr><td>HTML</td><td>2 h</td></tr>
        <tr><td>CSS</td><td>3 h</td></tr>
    </tbody>
    <tfoot>
        <tr><th scope="row">Total</th><td>5 h</td></tr>
    </tfoot>
</table>
```

**colspan** faz uma célula ocupar várias colunas. **rowspan** faz uma célula ocupar várias linhas. São úteis em dados agrupados, mas não são necessários na tabela anterior.

```
<!-- Uma célula ocupa as duas colunas da sua linha -->
<tr><td colspan="2">Inscrições abertas</td></tr>

<!-- Duas linhas: a primeira célula é partilhada -->
<tr><th rowspan="2" scope="rowgroup">Web</th><td>HTML</td></tr>
<tr><td>CSS</td></tr>
```

**ERRO FREQUENTE**
As células abrangidas por colspan ou rowspan não devem ser repetidas. Coloca estes fragmentos numa table, dentro do grupo de linhas adequado.

**EXPERIMENTA TU**
Acrescenta uma oficina de JavaScript e atualiza o total. Resultado: três linhas de dados, além do cabeçalho e do resumo.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| table / caption | Tabela / título | <caption>Horário</caption> |
| thead / tbody / tfoot | Agrupam linhas | <tbody>…</tbody> |
| tr / th / td | Linha / cabeçalho / dado | <tr><td>CSS</td></tr> |
| colspan / rowspan | Unem colunas / linhas | colspan="2" |

# Pedir informação ao utilizador

Um formulário reúne campos e ações. O HTML cria a interface; não guarda mensagens nem envia emails por si só. Nesta sebenta a versão final simula a submissão no navegador, sem transmitir dados.

| Elemento | Função e sintaxe | Pequeno exemplo / resultado |
|---|---|---|
| form | Agrupa campos: <form>…</form> | <form id="contacto">…</form>: formulário identificável. |
| label | Nome visível: <label for="id">…</label> | <label for="nome">Nome</label>: etiqueta do campo nome. |
| input | Campo vazio: <input type="…"> | <input id="nome" type="text">: caixa de texto. |
| textarea | Texto longo: <textarea>…</textarea> | <textarea rows="4"></textarea>: área com quatro linhas. |
| select | Escolha: <select>…</select> | <select name="tema">…</select>: lista de opções. |
| option | Opção: <option value="…">…</option> | <option value="html">HTML</option>: uma escolha. |
| button | Ação: <button type="…">…</button> | <button type="submit">Enviar</button>: submete o formulário. |

```
<label for="nome">Nome</label>
<input type="text" id="nome" name="nome" required>
```

**Resultado:** clicar na etiqueta coloca o foco no campo. O valor de for tem de coincidir com o id. Um placeholder é uma pista, não substitui a etiqueta.

**ATENÇÃO**
Num formulário real, action indica o destino e method define o método, por exemplo POST. É preciso um servidor ou serviço de receção e validação do lado do servidor. O Live Server apenas serve ficheiros.

**EXPERIMENTA TU**
Cria contacto.html com a estrutura mínima e um título. Acrescenta este par de label e input, dentro de form. Testa a navegação com a tecla Tab.

# Tipos de input e atributos

| Tipo | Exemplo | Resultado / utilização |
|---|---|---|
| text | <input type="text"> | Texto numa linha. |
| password | <input type="password"> | Oculta visualmente os caracteres; não os encripta. |
| email | <input type="email"> | Valida a forma básica de um email. |
| number | <input type="number" min="1" max="20"> | Número com limites. |
| date | <input type="date"> | Escolha de data; aspeto varia por navegador. |
| radio | <input type="radio" name="turno" value="manha"> | Uma escolha num grupo com o mesmo name. |
| checkbox | <input type="checkbox" name="avisos" value="sim"> | Opção independente, marcada ou não. |
| file | <input type="file" name="ficheiro"> | Seleciona um ficheiro; não o envia sozinho. |
| submit | <input type="submit" value="Enviar"> | Botão de submissão. |
| reset | <input type="reset" value="Repor"> | Repõe os valores iniciais; pode apagar trabalho. |

| Atributo | Significado |
|---|---|
| name / id | name identifica o dado submetido; id identifica o elemento no documento. |
| value | Valor inicial ou valor associado a uma opção. |
| placeholder | Pista dentro do campo; desaparece ao escrever. |
| required | Exige preenchimento válido antes da submissão normal. |
| checked | Marca inicialmente uma checkbox ou um radio. |
| min / max | Limites para tipos compatíveis, como number e date. |

**BOA PRÁTICA**
Dá uma etiqueta a cada campo. Agrupa radios com o mesmo name e valores diferentes. Em formulários completos, fieldset e legend ajudam a dar um nome ao grupo. Para enviar ficheiros, um formulário real com POST precisa de enctype="multipart/form-data".

# Formulário de contacto: demonstração

Coloca este exemplo em contacto.html. O botão é do tipo button nesta fase para evitar uma submissão sem destino. No capítulo de JavaScript será transformado num botão submit com tratamento do evento.

```
<form id="contacto">
    <p>
        <label for="nome">Nome</label>
        <input id="nome" name="nome" type="text" required>
    </p>
    <p>
        <label for="email">Email</label>
        <input id="email" name="email" type="email" required>
    </p>
    <p>
        <label for="tema">Tema</label>
        <select id="tema" name="tema">
            <option value="html">HTML</option>
            <option value="css">CSS</option>
        </select>
    </p>
    <p>
        <label for="mensagem">Mensagem</label>
        <textarea id="mensagem" name="mensagem"
                  rows="4" required></textarea>
    </p>
    <button type="button">Demonstração: ainda sem envio</button>
</form>
```

**Resultado:** dois campos de uma linha, uma lista, uma área de texto e um botão. O botão ainda não faz nada; a validação de submissão será testada quando usarmos submit.

**ERRO FREQUENTE**
required="false" continua a ativar required, porque é um atributo booleano: a presença conta. Para o desativar, remove o atributo.

**EXPERIMENTA TU**
Acrescenta uma checkbox para pedir informações sobre oficinas. Usa uma etiqueta associada. Não peças palavras-passe nem dados pessoais reais nos exercícios.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| label + id | Associa nome ao campo | for="email" / id="email" |
| name | Identifica o dado | name="email" |
| required | Campo obrigatório | <input required> |
| button type | Define a ação | type="button" / "submit" |

# Dar nomes às áreas da página

HTML semântico descreve o papel do conteúdo. Ajuda quem lê o código, as tecnologias de apoio e os motores de pesquisa. Trocar uma div por nav não cria automaticamente um menu bonito: isso será trabalho do CSS.

| Elemento | Função e sintaxe | Exemplo / resultado |
|---|---|---|
| header | Introdução: <header>…</header> | <header><h1>Clube Web</h1></header>: cabeçalho. |
| nav | Navegação: <nav>…</nav> | <nav><a href="index.html">Início</a></nav>: ligações principais. |
| main | Conteúdo principal: <main>…</main> | <main><h1>Sobre</h1></main>: área principal. |
| section | Grupo temático: <section>…</section> | <section><h2>Oficinas</h2>…</section>: tema com título. |
| article | Conteúdo autónomo: <article>…</article> | <article><h2>Nova oficina</h2>…</article>: notícia. |
| aside | Conteúdo complementar: <aside>…</aside> | <aside><h2>Recursos</h2>…</aside>: informação relacionada. |
| footer | Fecho / créditos: <footer>…</footer> | <footer> Clube Web </footer>: rodapé. |

## Comparar duas estruturas

```
<!-- Contentores sem significado específico -->
<div class="topo">Clube Web</div>
<div class="menu">...</div>
<div class="conteudo">...</div>

<!-- Áreas com um papel explícito -->
<header>Clube Web</header>
<nav>...</nav>
<main>...</main>
```

**BOA PRÁTICA**
Neste projeto usa um main visível por página. Uma section deve normalmente ter um título. Usa div quando apenas precisas de agrupar elementos para apresentação.

# Reorganizar o Clube Web

```
<header>
    <h1>Clube Web</h1>
    <p>Aprender, experimentar, partilhar.</p>
</header>
<nav aria-label="Navegação principal">
    <a href="index.html">Início</a>
    <a href="sobre.html">Sobre</a>
    <a href="contacto.html">Contacto</a>
</nav>
<main>
    <section>
        <h2>As nossas oficinas</h2>
        <p>HTML, CSS e JavaScript para principiantes.</p>
    </section>
    <article>
        <h2>Inscrições abertas</h2>
        <p>A próxima oficina começa na segunda-feira.</p>
    </article>
</main>
<footer>
    <p>Projeto educativo do Clube Web.</p>
</footer>
```

**Resultado:** o conteúdo continua simples, mas a estrutura já identifica cabeçalho, navegação, área principal e rodapé. aria-label dá um nome à navegação para tecnologias de apoio.

## Mini projeto 2 | Website sobre um tema

**Objetivo:** Construir três páginas interligadas.

**Ficheiros:** index.html, sobre.html, contacto.html e imagens/.

**Requisitos obrigatórios:** Menu igual nas três páginas; títulos próprios; lista, imagem, tabela e formulário distribuídos pelo website.

**Etapas:** 1. Escolhe o tema. 2. Reutiliza a estrutura semântica. 3. Acrescenta conteúdos. 4. Testa cada ligação.

**Resultado esperado:** É possível visitar todas as páginas e regressar ao início.

**DESAFIO**
Acrescenta um aside com ligações de consulta e explica por que é conteúdo complementar.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| header / footer | Introdução / fecho | <footer>…</footer> |
| nav / main | Navegação / principal | <main>…</main> |
| section / article | Tema / peça autónoma | <article>…</article> |
| aside | Complemento | <aside>…</aside> |

# Som e vídeo na página

| Elemento | Função e sintaxe | Exemplo / resultado |
|---|---|---|
| audio | Leitor de som: <audio controls>…</audio> | Controlos de reprodução de áudio. |
| video | Leitor de vídeo: <video controls>…</video> | Área de vídeo com controlos. |
| source | Fonte vazia: <source src="…" type="…"> | Indica um ficheiro e o respetivo tipo. |

```
<audio controls>
    <source src="media/apresentacao.mp3" type="audio/mpeg">
    O teu navegador não suporta este leitor de áudio.
</audio>
<p>Transcrição: Bem-vindo ao Clube Web...</p>

<video controls width="480">
    <source src="media/oficina.mp4" type="video/mp4">
    O teu navegador não suporta este leitor de vídeo.
</video>
```

**Resultado:** aparecem leitores; só reproduzem se os ficheiros existirem e o navegador suportar o formato e o codec. Cria a pasta media e coloca nela ficheiros teus ou autorizados. Não estão incluídos no ZIP.

O atributo controls apresenta os comandos ao utilizador. Evita reprodução automática. Uma transcrição ajuda no áudio; para vídeo com fala, acrescenta legendas sincronizadas, por exemplo com track e um ficheiro WebVTT.

```
<!-- Dentro de video, depois de source -->
<track kind="captions" src="media/legendas.vtt"
       srclang="pt" label="Português" default>
```

**EXPERIMENTA TU**
Grava uma apresentação curta sem dados pessoais, com autorização quando aplicável. Insere-a com audio e escreve a transcrição. Testa reproduzir e pausar.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| audio / video | Leitores | <audio controls>…</audio> |
| source | Ficheiro multimédia | type="video/mp4" |
| controls | Mostra comandos | <video controls> |
| track | Legendas / captions | kind="captions" |

# Uma página dentro de outra

## iframe | Incorpora outro documento

**Sintaxe:** `<iframe src="pagina.html"></iframe>`

```
<iframe src="sobre.html" title="Informação sobre o clube"
        width="450" height="250"></iframe>
```

**Resultado:** Uma área com a página sobre.html dentro da página atual. **Dica:** Inclui sempre um title que identifique o conteúdo incorporado.

## Exemplo externo: mapa

```
<iframe
  title="Mapa OpenStreetMap"
  src="https://www.openstreetmap.org/export/embed.html"
  width="450" height="300" loading="lazy">
</iframe>
```

Este endereço apresenta o mapa incorporável do OpenStreetMap. Para mostrar uma zona específica, usa **Partilhar > HTML** no serviço e copia o código de incorporação gerado, mantendo a atribuição.

**ATENÇÃO**
Incorpora apenas conteúdos de origem conhecida. Um serviço externo pode receber dados de ligação e usar cookies. Alguns websites bloqueiam iframe com políticas como CSP ou X-Frame-Options; não tentes contornar esse bloqueio.

O atributo **sandbox** permite restringir ações do documento incorporado, mas pode impedir o seu funcionamento. Para uma página local estática podes experimentar sandbox sem permissões; acrescenta apenas as permissões que compreendes e de que precisas.

**EXPERIMENTA TU**
Incorpora sobre.html e compara com uma hiperligação normal para a mesma página. Depois testa o mapa com ligação à Internet. Resultado: uma ligação navega; um iframe reserva uma área dentro da página.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| iframe | Documento incorporado | <iframe src="sobre.html">…</iframe> |
| title | Nome acessível | title="Mapa" |
| sandbox | Restrições | <iframe sandbox …> |
| loading | Adia carga quando possível | loading="lazy" |

# Separar conteúdo e apresentação

CSS é a linguagem que define a apresentação: cor, letra, espaçamento e disposição. HTML diz “isto é um título”; CSS decide como esse título é mostrado. Um ficheiro CSS pode ser partilhado por várias páginas.

## Três formas de aplicar estilos

```
<!-- 1. Inline: no próprio elemento -->
<p style="color: blue;">Texto azul</p>

<!-- 2. Interno: dentro de head -->
<style>
    p { color: blue; }
</style>

<!-- 3. Externo: ligação dentro de head -->
<link rel="stylesheet" href="css/estilo.css">
```

**Resultado:** os três métodos podem tornar o texto azul. Vamos usar **CSS externo**: evita repetição e permite alterar todo o website num só local. No ficheiro .css escreve regras CSS, sem tags style.

## Ler uma regra

```
seletor {
    propriedade: valor;
}

p {
    color: blue;
}
```

**p** seleciona os parágrafos; **color** é a propriedade; **blue** é o valor. O par propriedade: valor é uma declaração. As chavetas agrupam declarações e o ponto e vírgula separa-as.

**EXPERIMENTA TU**
Liga css/estilo.css às três páginas do Clube Web. Escreve p { color: blue; } nesse ficheiro. Confirma que todos os parágrafos ficam azuis.

**ERRO FREQUENTE**
Se o CSS não aparece, verifica o caminho de link, o nome do ficheiro e se guardaste. Não escrevas uma regra solta fora das tags style quando usas CSS interno.

# Experimentar, observar e corrigir

```
/* Estilos base do Clube Web */
body {
    font-family: Arial, sans-serif;
    color: #17324d;
    background-color: #f4f7fa;
}

h1 {
    color: #006b70;
}
```

**Resultado:** texto numa família sem serifa, fundo claro e título verde-azulado. Se Arial não existir, o navegador escolhe uma fonte da família genérica sans-serif.

Comentários CSS usam /* ... */. Podes abrir as ferramentas de programador com **F12** e inspecionar um elemento. Alterações feitas aí são temporárias; escreve a versão definitiva no ficheiro CSS.

**CORRIGE O CÓDIGO • 5 MINUTOS**
A regra p { color: blue font-size: 18px; } tem duas declarações sem separação. Corrige-a e confirma as duas propriedades no navegador.

**BOA PRÁTICA**
Termina cada declaração com ;, incluindo a última. A última pode ser válida sem esse sinal, mas mantê-lo facilita acrescentar propriedades.

**EXPERIMENTA TU**
Muda apenas a cor de h1. Depois muda apenas o fundo de body. Prevê o efeito antes de guardar. Regista qual seletor afeta cada parte.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| link | Liga CSS externo | href="css/estilo.css" |
| Seletor | Escolhe elementos | p { ... } |
| Declaração | Propriedade e valor | color: blue; |
| Comentário | Explicação no CSS | /* Menu principal */ |

# Escolher o que queres alterar

| Seletor | Exemplo | Seleciona |
|---|---|---|
| Elemento | p { color: blue; } | Todos os parágrafos. |
| Classe | .destaque { font-weight: bold; } | Elementos com class="destaque". |
| Identificador | #cabecalho { background: lightgray; } | O elemento com id="cabecalho". |
| Múltiplos | h1, h2 { color: teal; } | Todos os h1 e todos os h2. |
| Descendente | nav a { color: white; } | Ligações que estão dentro de nav. |

```
<header id="cabecalho">
    <h1>Clube Web</h1>
</header>
<p class="destaque">Inscrições abertas.</p>
<p>Oficinas para principiantes.</p>
<p class="destaque">Participação gratuita.</p>
```

```
.destaque {
    font-weight: bold;
}
#cabecalho {
    background-color: lightgray;
}
```

**Resultado:** dois parágrafos ficam a negrito e o cabeçalho ganha um fundo. **class** pode ser reutilizada. Um **id** deve identificar apenas um elemento em cada documento. Um elemento pode ter várias classes: class="cartao destaque".

**ERRO FREQUENTE**
No HTML escreve class="destaque", sem ponto. O ponto aparece no seletor CSS: .destaque. Da mesma forma, id="cabecalho" corresponde a #cabecalho.

**EXPERIMENTA TU**
Aplica a classe destaque a um parágrafo e a um item de lista. Confirma que a classe não depende do tipo de elemento.

# Quando duas regras entram em conflito

A **cascata** decide quais os valores aplicados. Neste nível, compara regras de autor normais, sem camadas nem !important: um seletor com id tem mais especificidade do que uma classe; uma classe tem mais do que um elemento.

```
p { color: blue; }
.aviso { color: darkred; }

/* Com a mesma especificidade, a última regra ganha. */
.aviso { color: darkgreen; }
```

Para Prazo amanhã , a cor será verde-escura. A classe vence p e, entre as duas regras .aviso, vence a última. Evita resolver tudo com !important: procura primeiro a regra que está a ganhar.

Algumas propriedades, como color e font-family, podem ser **herdadas** de um antepassado. Outras, como margin, não são herdadas normalmente. Definir a letra em body evita repeti-la em cada parágrafo.

**QUAL SERÁ O RESULTADO? • 5 MINUTOS**
Em nav a { color: red; }, muda também a cor de uma ligação no footer, fora do nav? Explica o espaço entre os dois seletores.

**EXPERIMENTA TU**
Cria duas regras para .destaque com cores diferentes. Troca a ordem. Depois inspeciona o elemento e identifica qual declaração ficou anulada.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| p | Todos os parágrafos | p { color: blue; } |
| .classe | Grupo reutilizável | .cartao { padding: 1rem; } |
| #id | Elemento identificado | #topo { color: teal; } |
| a, button | Seleção conjunta | a, button { … } |
| nav a | Descendentes | nav a { … } |

# Texto legível e com hierarquia

| Propriedade | Função | Exemplo |
|---|---|---|
| color | Cor do texto | color: #17324d; |
| font-family | Família de letra e alternativas | font-family: Arial, sans-serif; |
| font-size | Tamanho do texto | font-size: 1.1rem; |
| font-weight | Peso / espessura | font-weight: bold; |
| font-style | Estilo, como itálico | font-style: italic; |
| text-align | Alinhamento do texto | text-align: center; |
| text-decoration | Sublinhado e outras linhas | text-decoration: underline; |
| line-height | Distância entre linhas | line-height: 1.6; |

**px** é uma unidade CSS de dimensão. **rem** refere-se ao tamanho de letra do elemento raiz. Se esse tamanho for 16 px, 1.5rem corresponde a 24 px. **%** depende de uma referência, como a largura do contentor.

```
body {
    font-family: Arial, sans-serif;
    font-size: 1rem;
    line-height: 1.6;
    color: #17324d;
}
h1 {
    font-size: 2rem;
    font-weight: bold;
}
.nota { font-style: italic; }
```

**Resultado:** texto com espaço entre linhas e título destacado. O valor 1.6 de line-height é um multiplicador do tamanho de letra, não 1.6 píxeis.

**EXPERIMENTA TU**
Aumenta line-height de 1 para 1.6 num parágrafo longo. Compara a leitura. Escolhe cores com contraste e evita blocos longos de texto centrado.

**BOA PRÁTICA**
Mantém as ligações reconhecíveis e o foco do teclado visível. Não retires sublinhados e contornos sem uma alternativa igualmente clara.

# Fundos e dimensões

| Propriedade | Função | Exemplo |
|---|---|---|
| background-color | Cor de fundo | background-color: #eef7f7; |
| background-image | Imagem decorativa de fundo | background-image: url("../imagens/escola.jpg"); |
| background-size | Escala do fundo | background-size: cover; |
| background-position | Posição do fundo | background-position: center; |
| width / height | Largura / altura | width: 100%; height: auto; |
| max-width | Limite máximo | max-width: 1000px; |
| min-width | Limite mínimo | min-width: 180px; |

```
.capa {
    background-color: #e4f2f2;
    background-image: url("../imagens/escola.jpg");
    background-size: cover;
    background-position: center;
    min-height: 180px;
}
main {
    width: 90%;
    max-width: 1000px;
    margin: 0 auto;
}
```

**Resultado:** a capa ocupa pelo menos 180 px de altura e o conteúdo principal cresce até 1000 px. margin: 0 auto centra horizontalmente um bloco com largura inferior ao contentor.

Em CSS externo, url(...) é relativo ao **ficheiro CSS**. Como estilo.css está em css/, ../imagens sobe uma pasta. cover preenche a área e pode cortar partes da imagem; contain mostra-a inteira e pode deixar espaço vazio.

**ERRO FREQUENTE**
Não uses uma altura fixa pequena para uma caixa com muito texto. Pode haver transbordo quando o utilizador amplia a letra. Evita também min-width demasiado grande em ecrãs pequenos.

**EXPERIMENTA TU**
Define max-width no main e reduz a janela. Depois testa cover e contain num fundo. Mantém informação importante em texto ou em img com alt, não apenas no fundo.

# Espaçamento, bordas e cartões

| Propriedade | Função | Exemplo |
|---|---|---|
| margin | Espaço exterior | margin: 1rem 0; |
| padding | Espaço entre conteúdo e borda | padding: 1rem; |
| border | Espessura, estilo e cor | border: 1px solid #b9cbd5; |
| border-radius | Arredonda os cantos | border-radius: 12px; |

```
.cartao {
    background-color: white;
    padding: 1rem;
    margin: 1rem 0;
    border: 1px solid #b9cbd5;
    border-radius: 12px;
}
```

**Resultado:** cada elemento com class="cartao" tem fundo branco, espaço interior, separação vertical e cantos arredondados. Em margin: 1rem 0, o primeiro valor é vertical e o segundo horizontal. Com quatro valores: cima, direita, baixo, esquerda.

## Mini projeto 3 | Página com CSS

**Objetivo:** Dar uma identidade visual ao Clube Web.

**Ficheiros:** As três páginas anteriores e css/estilo.css.

**Requisitos obrigatórios:** CSS externo; duas cores principais; letra legível; margens; pelo menos três cartões.

**Etapas:** 1. Liga o CSS. 2. Define estilos de body e títulos. 3. Cria a classe cartao. 4. Revê o contraste e o espaço.

**Resultado esperado:** As três páginas partilham um aspeto coerente.

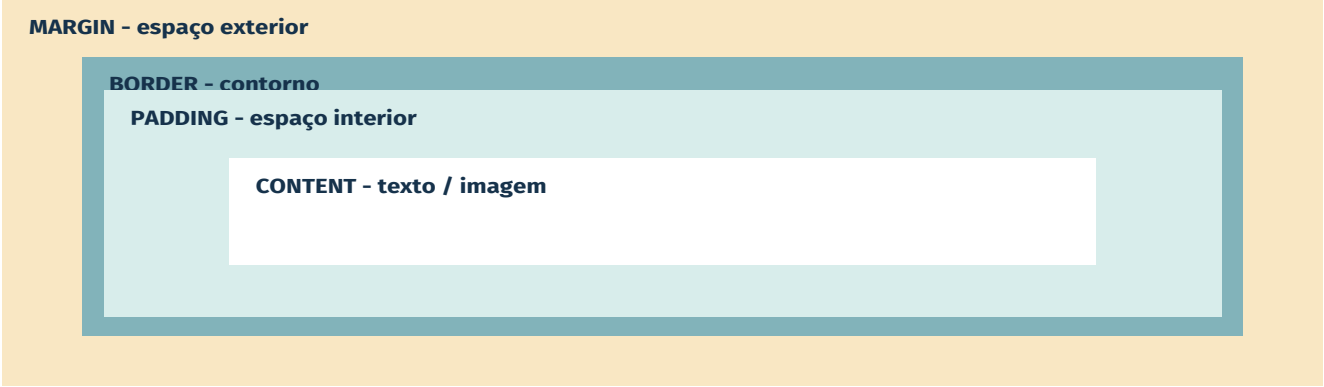
**DESAFIO**
Cria uma classe cartao-destaque que se possa combinar com cartao sem repetir todas as propriedades.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| Texto | Cor, letra e leitura | line-height: 1.6; |
| Fundo | Cor / imagem | background-color: white; |
| Dimensão | Tamanho e limites | max-width: 1000px; |
| Espaçamento | Interior / exterior | padding: 1rem; |
| Borda | Contorno / cantos | border-radius: 12px; |

# Cada elemento é uma caixa

O modelo de caixa explica o espaço ocupado por um elemento. Do interior para o exterior: **content** (conteúdo), **padding**, **border** e **margin**.




```
.caixa {
    width: 240px;
    padding: 20px;
    border: 2px solid teal;
    margin: 10px;
    box-sizing: border-box;
}
```

**Com content-box**, o valor inicial de box-sizing, width aplica-se ao conteúdo: a largura até à borda é 240 + 40 + 4 = **284 px**. Com as margens horizontais, ocupa 304 px.

**Com border-box**, os 240 px já incluem padding e border. O conteúdo fica com 196 px (240 - 40 - 4). As margens continuam de fora: largura total com margens = **260 px**.

```
/* Uma base habitual para o website inteiro */
*, *::before, *::after {
    box-sizing: border-box;
}
```

O asterisco seleciona todos os elementos; before e after são pseudo-elementos.

**EXPERIMENTA TU**
Cria a caixa do exemplo e inspeciona o Box Model nas ferramentas do navegador. Liga e desliga border-box; compara as medidas. Depois altera apenas padding.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| content | Texto / imagem | width depende de box-sizing |
| padding / border | Espaço interno / contorno | padding: 20px; |
| margin | Espaço exterior | margin: 10px; |
| border-box | Inclui padding e borda | box-sizing: border-box; |

# Um menu numa direção

Flexbox organiza os filhos diretos de um contentor segundo um eixo principal. É útil para menus, barras de ações e grupos de cartões. Por predefinição, esse eixo é uma linha horizontal.

```
<nav class="menu" aria-label="Navegação principal">
    <a href="index.html">Início</a>
    <a href="sobre.html">Sobre</a>
    <a href="contacto.html">Contacto</a>
</nav>
```

```
.menu {
    display: flex;
    flex-direction: row;
    justify-content: center;
    align-items: center;
    gap: 16px;
    flex-wrap: wrap;
}
```

| Propriedade | Função |
|---|---|
| display: flex | Ativa Flexbox no contentor; os filhos diretos tornam-se itens flex. |
| flex-direction | row: linha; column: coluna. Define o eixo principal. |
| justify-content | Distribui os itens no eixo principal, por exemplo center ou space-between. |
| align-items | Alinha no eixo transversal, por exemplo center ou stretch. |
| gap | Cria espaço entre os itens. |
| flex-wrap | wrap permite novas linhas quando falta espaço. |

**Resultado:** as ligações ficam lado a lado, centradas, com 16 px entre elas. Numa janela estreita podem passar para outra linha.

**EXPERIMENTA TU**
Troca row por column. Observa que justify-content passa a atuar na direção vertical. Não memorizes “justify é horizontal”: depende da direção.

# Cartões, alinhamento e galeria

```
<div class="cartoes">
    <article class="cartao"><h2>HTML</h2><p>Estrutura.</p></article>
    <article class="cartao"><h2>CSS</h2><p>Apresentação.</p></article>
    <article class="cartao"><h2>JS</h2><p>Interação.</p></article>
</div>
```

```
.cartoes {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;
    align-items: stretch;
}
.cartoes > .cartao {
    flex: 1 1 220px;
    margin: 0;
}
```

**Resultado:** os cartões partilham o espaço disponível e mudam de linha quando necessário. flex: 1 1 220px significa: pode crescer, pode encolher e parte de uma base de 220 px. O símbolo > seleciona filhos diretos.

Para uma galeria simples, coloca uma imagem dentro de cada cartão e aplica max-width: 100%; height: auto às imagens. Para centrar um ícone e texto numa linha, usa display: flex; align-items: center; gap: 8px no seu contentor.

**EXPERIMENTA TU**
Acrescenta um quarto cartão. Testa a 1000 px e a 400 px de largura. Resultado: os cartões continuam legíveis e passam para novas linhas.

**MODIFICA O EXEMPLO • 10 MINUTOS**
Altera justify-content no menu entre flex-start, center e space-between. Desenha o resultado de cada opção antes de testar.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| display | Ativa Flexbox | display: flex; |
| Direção | Eixo principal | flex-direction: column; |
| Alinhamento | Principal / transversal | justify-content: center; |
| Quebra / espaço | Novas linhas / distância | flex-wrap: wrap; gap: 20px; |
| flex | Crescer, encolher, base | flex: 1 1 220px; |

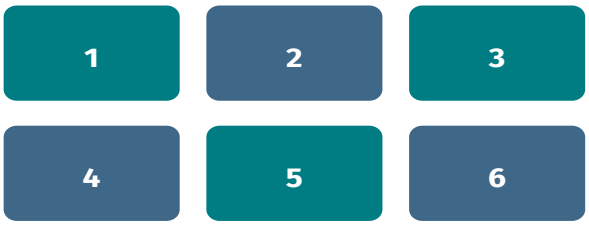
# Linhas e colunas em conjunto

Grid organiza elementos em duas dimensões: linhas e colunas. É útil quando queres alinhar cartões numa grelha. Flexbox organiza sobretudo a distribuição numa direção, mesmo quando permite várias linhas.

**Flexbox: uma direção**




**Grid: linhas e colunas**




```
<div class="grelha">
    <article class="cartao">Oficina de HTML</article>
    <article class="cartao">Oficina de CSS</article>
    <article class="cartao">Oficina de JavaScript</article>
</div>
```

```
.grelha {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 20px;
}
.grelha > .cartao { margin: 0; }
```

**Resultado:** três colunas iguais. fr representa uma fração do espaço disponível; repeat(3, 1fr) repete essa dimensão três vezes. Um quarto cartão começa uma nova linha.

**EXPERIMENTA TU**
Cria seis cartões, cada um com título e imagem. Testa duas, três e quatro colunas. Não uses este exemplo fixo em telemóveis sem a adaptação do próximo capítulo.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| display: grid | Ativa a grelha | display: grid; |
| grid-template-columns | Define colunas | repeat(3, 1fr) |
| gap | Espaço entre células | gap: 20px; |
| fr | Fração disponível | 1fr 2fr |

# Uma página, vários ecrãs

Uma página responsiva adapta a disposição ao espaço disponível. O utilizador pode estar num computador, tablet ou smartphone, ampliar o texto ou reduzir a janela. Evita obrigá-lo a deslocar toda a página na horizontal.

```
<!-- Dentro de head, em todas as páginas -->
<meta name="viewport"
      content="width=device-width, initial-scale=1.0">
```

O viewport informa o navegador móvel para utilizar a largura do dispositivo. Não torna uma página responsiva sozinho: também precisas de dimensões flexíveis e regras de disposição.

```
img {
    max-width: 100%;
    height: auto;
}

@media (max-width: 768px) {
    .grelha {
        grid-template-columns: 1fr;
    }
}
```

**Resultado:** imagens não ultrapassam o contentor e preservam a proporção. A grelha do capítulo anterior passa de três colunas para uma quando o viewport tem 768 px ou menos. A media query só aplica as regras quando a condição é verdadeira.

**BOA PRÁTICA**
Escolhe pontos de mudança quando o conteúdo deixa de caber bem, não apenas por nomes de dispositivos. Flexbox com wrap resolve muitos casos sem media queries.

**EXPERIMENTA TU**
Abre o modo de dispositivos nas ferramentas do navegador. Testa 360, 768 e 1200 px, e amplia o texto. Verifica menu, imagens, formulário e tabela.

**ERRO FREQUENTE**
Não desatives o zoom com user-scalable=no. Não uses width: 1000px num contentor que precisa de caber num telemóvel.

# Construir a partir do ecrã pequeno

Na abordagem **mobile first**, a base funciona num ecrã estreito. Acrescentas colunas quando existe espaço. Este exemplo **substitui** as regras de colunas anteriores; não juntes estratégias sem perceber a cascata.

```
.grelha {
    display: grid;
    grid-template-columns: 1fr;
    gap: 20px;
}
@media (min-width: 600px) {
    .grelha { grid-template-columns: repeat(2, 1fr); }
}
@media (min-width: 960px) {
    .grelha { grid-template-columns: repeat(3, 1fr); }
}
```

**Resultado:** uma coluna abaixo de 600 px, duas de 600 a 959 px e três a partir de 960 px. Uma tabela larga pode ter um contentor com overflow-x: auto, sem provocar deslocamento horizontal da página inteira.

## Mini projeto 4 | Galeria responsiva

**Objetivo:** Organizar seis imagens com legendas.

**Ficheiros:** index.html, css/estilo.css e seis imagens autorizadas.

**Requisitos obrigatórios:** Grid ou Flexbox; seis cartões; alt adequado; uma coluna em ecrã estreito.

**Etapas:** 1. Cria os cartões. 2. Aplica a grelha. 3. Limita as imagens. 4. Testa três larguras.

**Resultado esperado:** As imagens mantêm proporção e não saem do ecrã.

**DESAFIO**
Usa figure e figcaption para associar cada imagem à legenda.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| viewport | Largura no telemóvel | width=device-width |
| @media | Regra condicional | @media (min-width: 600px) |
| Imagem fluida | Limita sem deformar | max-width: 100%; |
| Grid + Flexbox | Grelha / navegação | 1fr / flex-wrap: wrap |

# Mini projeto: website adaptável

## Mini projeto 5 | Website responsivo

**Objetivo:** Adaptar todas as páginas do Clube Web.

**Ficheiros:** index.html, sobre.html, contacto.html, css/estilo.css e imagens/.

**Requisitos obrigatórios:** Estrutura semântica; menu Flexbox; cartões Grid; formulário utilizável; imagens fluidas; três larguras testadas.

**Etapas:** 1. Começa pela largura de 360 px. 2. Corrige o que transborda. 3. Acrescenta colunas com min-width. 4. Testa 768 e 1200 px. 5. Testa teclado e zoom.

**Resultado esperado:** Navegação, conteúdo e formulário funcionam em ecrãs estreitos e largos.

**DESAFIO**
Testa também a 200% de zoom e cria uma ligação “Saltar para o conteúdo” visível quando recebe foco.

## Ficha de teste do aluno

| Verificação | Como testar | Critério de sucesso |
|---|---|---|
| Menu | Reduzir a janela | Ligações legíveis; podem mudar de linha. |
| Imagens | Comparar proporções | Sem esticar ou ultrapassar o contentor. |
| Texto | Ampliar para 200% | Conteúdo legível e acessível. |
| Formulário | Usar Tab e escrever | Etiquetas visíveis; campos cabem. |
| Tabela | Ver no ecrã pequeno | Dados acessíveis, com deslocamento local se preciso. |
| Teclado | Tab, Shift+Tab e Enter | Foco visível e ordem compreensível. |

**ANTES DE AVANÇAR**
Já consegues criar conteúdo, apresentar esse conteúdo e adaptá-lo ao ecrã? Se sim, está na altura de acrescentar comportamento com JavaScript.

# Acrescentar comportamento

JavaScript permite responder a ações: mostrar uma mensagem, alterar uma imagem ou validar uma interação. HTML continua a estruturar o conteúdo e CSS continua a apresentá-lo.

```
<!-- No fim de body, antes de </body> -->
<script src="js/script.js"></script>
```

Colocar o script no fim de body permite que os elementos anteriores já existam quando o código é executado. Outra opção é colocar a ligação no head com o atributo **defer**. Escolhe uma das opções, sem carregar o mesmo ficheiro duas vezes.

```
// Ficheiro js/script.js
function mostrarMensagem() {
    alert("Olá!");
}
```

```
<!-- Dentro de body -->
<button type="button" onclick="mostrarMensagem()">
    Clica aqui
</button>
```

**Resultado:** clicar abre uma caixa de diálogo com “Olá!”. function define uma função, ou seja, um conjunto de instruções com nome. Os parênteses em mostrarMensagem() chamam a função.

**DICA**
onclick é usado aqui para tornar a ligação entre botão e função evidente. Nos exemplos seguintes vamos separar o comportamento do HTML com addEventListener. Usa alert apenas em exercícios curtos; interrompe a interação.

**EXPERIMENTA TU**
Cria script.js e liga-o à página inicial. Testa o botão. Altera a mensagem. Se não funcionar, abre F12 > Console e lê o primeiro erro.

**ERRO FREQUENTE**
JavaScript distingue maiúsculas e minúsculas: mostrarMensagem e mostrarmensagem são nomes diferentes. No ficheiro .js não escrevas tags script.

# Variáveis, valores e operadores

Uma variável associa um nome a um valor. Usa **const** quando não vais reatribuir esse nome e **let** quando o valor terá de mudar. Strings são texto entre aspas; números escrevem-se sem aspas.

```
const clube = "Clube Web";       // string
let inscritos = 12;              // número
const aberto = true;             // booleano: true ou false

inscritos = inscritos + 1;
console.log(clube);
console.log(inscritos);          // 13
```

| Operador | Significado | Exemplo / resultado |
|---|---|---|
| + - * / | Operações numéricas | 6 * 2 resulta em 12. |
| + com strings | Junta texto | "Olá, " + "Ana" resulta em "Olá, Ana". |
| = | Atribuição | inscritos = 13; |
| === / !== | Igualdade / diferença estritas | 5 === "5" resulta em false. |
| > < >= <= | Comparações | 13 >= 10 resulta em true. |
| && / \|\| / ! | E / OU / NÃO lógicos | aberto && inscritos < 20 |

console.log mostra valores na consola do navegador, não na página. O valor de um campo de formulário é normalmente uma string: para cálculos, converte de forma explícita e valida o resultado.

```
const texto = "12";
console.log(texto + 1);          // "121"
console.log(Number(texto) + 1);  // 13
```

**QUAL SERÁ O RESULTADO? • 5 MINUTOS**
Prevê os resultados de 4 + 2, "4" + 2 e 4 === "4". Testa na consola e explica a diferença.

**EXPERIMENTA TU**
Cria const lotacao = 20 e let inscritos = 12. Calcula as vagas com subtração e mostra o resultado na consola.

# Condições e funções

Uma condição escolhe um caminho. O bloco if é executado quando a expressão é verdadeira; else trata o outro caso.

```
const vagas = 3;
if (vagas > 0) {
    console.log("Ainda há vagas.");
} else {
    console.log("Inscrições encerradas.");
}
```

**Resultado:** aparece “Ainda há vagas.”. Com vagas = 0, aparece a outra mensagem. As chavetas agrupam as instruções de cada ramo.

```
function calcularVagas(lotacao, inscritos) {
    return lotacao - inscritos;
}

const livres = calcularVagas(20, 12);
console.log(livres); // 8
```

lotacao e inscritos são **parâmetros**: nomes usados dentro da função. 20 e 12 são os valores passados na chamada. return devolve o resultado. A função pode ser reutilizada com outros valores.

**ERRO FREQUENTE**
Não confundas = com ===. A primeira forma atribui um valor; a segunda compara. Para escrever uma condição de igualdade usa, por exemplo, vagas === 0.

**EXPERIMENTA TU**
Cria uma função criarSaudacao(nome) que devolva "Olá, " + nome. Chama-a com dois nomes fictícios e mostra os resultados.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| const / let | Guardar valores | let total = 0; |
| String / número | Texto / cálculo | "12" / 12 |
| if / else | Escolher um caminho | if (vagas > 0) { … } |
| function / return | Reutilizar / devolver | return a + b; |
| Evento | Uma ação detetada | click / submit |

# Encontrar elementos e alterar texto

O **DOM** é a representação do documento como uma árvore de objetos. JavaScript pode encontrar um elemento e alterar o seu texto, atributos ou classes. O ficheiro HTML no disco não é reescrito por estas alterações.

```
<p id="mensagem">Bem-vindo ao clube.</p>
<button id="saudar" type="button">Mostrar mensagem</button>
```

```
const mensagem = document.getElementById("mensagem");
const botao = document.querySelector("#saudar");

function saudar() {
    mensagem.textContent = "O texto foi alterado!";
}

botao.addEventListener("click", saudar);
```

**Resultado:** o texto só muda depois do clique. getElementById recebe o id sem #. querySelector recebe um seletor CSS e devolve o primeiro elemento correspondente. Ambos podem devolver null se nada corresponder.

addEventListener associa uma função a um evento. Escrevemos saudar sem parênteses para entregar a função, em vez de a executar logo. O evento click também é acionado pela ativação de um botão com o teclado.

**ERRO FREQUENTE**
Se o mesmo script é usado em várias páginas, um botão pode não existir em todas. Verifica a existência antes de lhe associar eventos:

```
if (botao && mensagem) {
    botao.addEventListener("click", saudar);
}
```

Esta versão protegida substitui a associação anterior; não uses as duas simultaneamente.

**EXPERIMENTA TU**
Acrescenta um segundo botão que devolve a mensagem original. Usa funções com nomes claros. Resultado: consegues alternar entre os dois textos.

# Alterar uma imagem e uma classe

## Trocar uma imagem

```
<img id="foto" src="imagens/oficina.svg"
     alt="Ilustração de uma oficina Web">
<button id="trocar" type="button">Ver outra imagem</button>
```

```
const foto = document.querySelector("#foto");
const trocar = document.querySelector("#trocar");

if (foto && trocar) {
    trocar.addEventListener("click", function () {
        foto.src = "imagens/equipa.svg";
        foto.alt = "Ilustração da equipa do clube";
    });
}
```

**Resultado:** muda a imagem e a descrição. Os dois ficheiros têm de existir; estão disponíveis na pasta imagens da versão final de apoio. O caminho atribuído é resolvido relativamente ao documento HTML.

## Mudar o aspeto através de uma classe

```
/* Em css/estilo.css */
.realce { background-color: #fff1b8; }
```

```
<!-- No HTML -->
<p id="aviso">A próxima oficina é gratuita.</p>
<button id="realcar" type="button">Alternar realce</button>
```

```
const aviso = document.querySelector("#aviso");
const realcar = document.querySelector("#realcar");
if (aviso && realcar) {
    realcar.addEventListener("click", function () {
        aviso.classList.toggle("realce");
    });
}
```

**Resultado:** um clique acrescenta realce; o seguinte retira-o. JavaScript muda a classe e CSS define a aparência.

**EXPERIMENTA TU**
Adapta realce para incluir uma borda. Confirma que não precisaste de alterar o JavaScript para mudar a apresentação.

# Mostrar e esconder conteúdo

```
<button id="alternar" type="button"
        aria-expanded="false" aria-controls="detalhes">
    Mostrar detalhes
</button>
<p id="detalhes" hidden>As oficinas decorrem na sala 12.</p>
```

```
const alternar = document.querySelector("#alternar");
const detalhes = document.querySelector("#detalhes");

if (alternar && detalhes) {
    alternar.addEventListener("click", function () {
        detalhes.hidden = !detalhes.hidden;
        alternar.setAttribute("aria-expanded",
            String(!detalhes.hidden));
        if (detalhes.hidden) {
            alternar.textContent = "Mostrar detalhes";
        } else {
            alternar.textContent = "Esconder detalhes";
        }
    });
}
```

**Resultado:** o texto começa escondido. Cada clique inverte hidden, atualiza o texto do botão e informa se o conteúdo está expandido. ! inverte um booleano. String converte o valor para texto no atributo.

**BOA PRÁTICA**
Usa um botão para uma ação e uma ligação para navegar. aria-controls identifica o conteúdo controlado; aria-expanded comunica o estado. Não apliques display: block ao elemento hidden, pois podes contrariar a ocultação.

**EXPERIMENTA TU**
Acrescenta uma segunda informação extra com outro id. Não repitas identificadores. Testa os botões com rato e com teclado.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| getElementById | Procura um id | getElementById("aviso") |
| querySelector | Primeiro seletor | querySelector(".cartao") |
| textContent | Texto simples | elemento.textContent = "Olá"; |
| classList.toggle | Alterna uma classe | classList.toggle("realce") |
| hidden | Mostra / esconde | elemento.hidden = true; |

# Responder ao formulário

Recupera o formulário do capítulo 5. Substitui o seu botão por um submit e acrescenta uma área de resposta depois de form:

```
<!-- Dentro do formulário -->
<button type="submit">Testar mensagem</button>

<!-- Depois de </form> -->
<p id="resposta" role="status"></p>
```

```
const formulario = document.querySelector("#contacto");
const resposta = document.querySelector("#resposta");

if (formulario && resposta) {
    formulario.addEventListener("submit", function (evento) {
        evento.preventDefault();
        const nome = document.querySelector("#nome").value.trim();
        resposta.textContent = "Obrigado, " + nome +
            ". Demonstração concluída; nada foi enviado.";
    });
}
```

**Resultado:** o navegador verifica os campos required e o formato básico do email. Quando o formulário é válido, o evento é tratado sem navegar para outra página. preventDefault impede a submissão normal. trim remove espaços no início e no fim.

role="status" ajuda tecnologias de apoio a anunciar a resposta. textContent trata o nome como texto, sem o interpretar como HTML. O exemplo não faz validação de negócio, não guarda dados e não envia emails.

## Mini projeto 6 | Página interativa

**Objetivo:** Acrescentar dois comportamentos ao website.

**Ficheiros:** Páginas, css/estilo.css e js/script.js.

**Requisitos obrigatórios:** Botão mostrar/esconder; resposta de demonstração ao formulário; mensagens claras; ausência de erros na consola.

**Etapas:** 1. Liga o script. 2. Seleciona os elementos. 3. Associa eventos. 4. Testa campos inválidos e válidos.

**Resultado esperado:** A interface reage ao utilizador sem recarregar a página.

**DESAFIO**
Acrescenta um contador de cliques e um botão para o repor a zero.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| submit | Evento do formulário | addEventListener("submit", ...) |
| preventDefault | Impede a ação normal | evento.preventDefault(); |

# Do primeiro título ao website completo

O Clube Web serve de fio condutor. Não precisas de recomeçar em cada capítulo: guarda uma cópia antes de uma alteração grande e melhora a mesma base.

| Etapa | Acrescenta ao projeto | Confirma antes de avançar |
|---|---|---|
| 1. Texto | h1, h2 e p | Título no separador e na página. |
| 2. Listas | Interesses e sequência de trabalho | ul ou ol conforme o significado. |
| 3. Imagens | Imagem do clube | Caminho válido e alt. |
| 4. Ligações | Destinos internos e externos | Cada ligação abre o destino correto. |
| 5. Várias páginas | sobre.html e contacto.html | Menu permite ir e regressar. |
| 6. Tabela | Oficinas e durações | Cabeçalhos e total corretos. |
| 7. Formulário | Nome, email, tema e mensagem | Etiquetas associadas. |
| 8. Semântica | header, nav, main e footer | Áreas com papéis claros. |
| 9. CSS | Cores, texto e espaços | Mesmo CSS nas três páginas. |
| 10. Menu | Aspeto e foco das ligações | Página atual reconhecível. |
| 11. Flexbox | Menu flexível | Ligações mudam de linha. |
| 12. Grid | Cartões em grelha | Colunas alinhadas. |
| 13. Responsividade | Imagens fluidas e media queries | Testes a 360, 768 e 1200 px. |
| 14. JavaScript | Botões e formulário simulado | Interação e consola sem erros. |

**FICHEIROS DE APOIO**
01-html: versão sem estilos. 02-css: mesma estrutura com apresentação responsiva. 03-final: acrescenta interatividade. Compara index.html entre versões. As três pastas são independentes; abre apenas uma de cada vez no editor.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| HTML | Começa pelo conteúdo | 01-html/ |
| CSS | Acrescenta apresentação | 02-css/ |
| JavaScript | Acrescenta comportamento | 03-final/ |

# Observar o resultado no navegador

Estas capturas mostram a versão final do Clube Web. O conteúdo é o mesmo: a disposição adapta-se à largura. Os recortes mostram o topo das páginas, não toda a sua altura.

**Computador: 1200 px**




**Telemóvel: 360 px**




**EXPERIMENTA TU**

Localiza o menu Flexbox e os cartões Grid. Explica por que existem três colunas no computador e apenas uma no telemóvel. Abre o exemplo e confirma o ponto de mudança no CSS.

# Desafios de 5 a 15 minutos

**A • COMPLETA O CÓDIGO • 5 MINUTOS**
Completa os espaços para ligar o CSS externo e o script. Escreve as linhas no local correto do documento.

```
<link rel="________" href="css/estilo.css">
<script ____="js/script.js"></script>
```

**B • ENCONTRA O ERRO • 5 MINUTOS**
O HTML tem class="cartao", mas a borda não aparece. Identifica o erro no seletor e corrige-o.

```
#cartao { border: 1px solid teal; }
```

**C • QUAL SERÁ O RESULTADO? • 5 MINUTOS**
Uma grelha tem repeat(3, 1fr) e cinco cartões. Quantos ficam na segunda linha? Responde antes de testar.

**D • CORRIGE O CÓDIGO • 10 MINUTOS**
Pretendemos escrever “Olá” no parágrafo com id="mensagem". Existem dois erros na seleção/alteração.

```
const texto = document.getElementById("#mensagem");
texto.textcontent = "Olá";
```

**E • MODIFICA O EXEMPLO • 15 MINUTOS**
Cria um botão e um parágrafo. Usa let para contar cliques. Em cada click, incrementa o contador e mostra o total com textContent. Não uses alert.

**EXPERIMENTA TU**
Escolhe dois desafios. Para cada um, regista: previsão, resultado observado e explicação. Só depois consulta as soluções.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| Ler o erro | Identificar causa | F12 > Console |
| Isolar | Mudar uma coisa | Seletor antes da cor |
| Testar | Comparar expectativa | Clique / largura / campo |

# Uma rotina para escrever e corrigir

| Boa prática | Como aplicar |
|---|---|
| Indentação | Avança os elementos internos; mantém o mesmo estilo em todos os ficheiros. |
| Nomes e pastas | Minúsculas, sem espaços; html na raiz, estilos em css, scripts em js. |
| Comentários úteis | Explica a intenção ou uma decisão, em vez de repetir cada linha. |
| Semântica | Escolhe o elemento pelo papel do conteúdo. |
| Imagens | Alt informativo; alt vazio em imagens apenas decorativas; ficheiros autorizados. |
| Separação | HTML para conteúdo, CSS externo para estilos, JavaScript externo para comportamento. |
| Simplicidade | Nomes claros, exemplos pequenos e uma alteração de cada vez. |
| Acessibilidade | Etiquetas nos campos, contraste suficiente, teclado e foco visível. |
| Cópias de trabalho | Guarda uma versão que funciona antes de grandes alterações. |

## Quando o resultado não é o esperado

**1.** Guarda o ficheiro. **2.** Confirma a página e a pasta que abriste. **3.** Verifica caminhos e nomes. **4.** Lê a consola. **5.** Inspeciona os estilos. **6.** Reduz o exemplo até encontrares a causa.

**ERRO FREQUENTE**
IDs repetidos tornam as relações e seleções ambíguas. Chavetas em falta podem estragar regras CSS. Um ; em falta entre declarações pode impedir a sua interpretação. Em JavaScript, escreve os nomes com a capitalização exata.

Podes validar o HTML com o Nu HTML Checker. A validação automática ajuda a encontrar erros de marcação, mas não substitui testar a utilização e a acessibilidade.

**EXPERIMENTA TU**
Pede a um colega para usar o teu website sem explicações. Regista três dificuldades e corrige pelo menos uma. Não alteres tudo ao mesmo tempo.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| HTML | Estrutura e relações | Tags / id / alt / for |
| CSS | Seleção e cascata | Classes / chavetas / ; |
| JavaScript | Valores e eventos | Consola / null / click |
| Teste | Uso real | Teclado / zoom / ecrã |

# O teu website, do plano à entrega

**Objetivo:** criar um pequeno website responsivo com um tema à tua escolha: associação, desporto, música, património local, clube escolar ou portefólio. O Clube Web é um exemplo, não um modelo para copiar sem compreender.

## Ficheiros obrigatórios

```
meu_site/
  index.html
  sobre.html       # podes escolher outro nome
  contacto.html    # podes escolher outro nome
  css/estilo.css
  js/script.js
  imagens/
```

## Requisitos obrigatórios

**Conteúdo:** três páginas interligadas; títulos e parágrafos; uma lista; imagens; ligações internas e externas; uma tabela de dados; um formulário com etiquetas.

**Estrutura:** cabeçalho, menu, área principal e rodapé; HTML semântico; lang, charset, viewport e títulos de página adequados.

**Apresentação:** CSS externo; cores e tipografia coerentes; Flexbox e/ou Grid; imagens fluidas; adaptação a computador, tablet e smartphone.

**Comportamento:** JavaScript externo e pelo menos uma funcionalidade observável, como mostrar/esconder detalhes, alterar uma imagem ou responder ao formulário. O envio de dados pode ser apenas simulado, desde que esteja claramente indicado.

## Etapas de execução

**1. Planear:** define público, objetivo e conteúdos. Desenha as três páginas em papel.
**2. Estruturar:** cria HTML e ligações; testa sem CSS.
**3. Apresentar:** aplica estilos, espaços e disposição.
**4. Adaptar:** testa ecrãs e zoom.
**5. Interagir:** acrescenta JavaScript e mensagens.
**6. Rever:** testa, corrige e prepara a entrega.

**DESAFIO**
Acrescenta uma funcionalidade tua, como um filtro simples de cartões. Explica o evento utilizado, os elementos alterados e a forma como testaste.

# Testar e apresentar o trabalho

## Lista de verificação antes da entrega

| Teste | Resultado esperado |
|---|---|
| Abrir index.html pelo Live Server | Página inicial completa; recursos carregados. |
| Percorrer o menu nas três páginas | Nenhuma ligação interna partida. |
| Testar imagens e ligação externa | Recursos corretos e textos alternativos adequados. |
| Inspecionar a tabela | Dados coerentes, caption e cabeçalhos. |
| Submeter formulário vazio e válido | Erros compreensíveis; resposta clara; simulação identificada. |
| Ativar a funcionalidade JavaScript | Comportamento previsível; sem erros na consola. |
| Testar 360 / 768 / 1200 px | Sem transbordo horizontal da página. |
| Usar teclado e zoom de 200% | Foco visível; conteúdo acessível. |
| Abrir uma cópia noutra pasta | Não depende de caminhos absolutos do computador. |

## Proposta de avaliação

| Critério | Peso |
|---|---|
| Estrutura HTML, semântica e conteúdo | 25% |
| CSS e organização visual | 20% |
| Responsividade | 20% |
| JavaScript e clareza das interações | 15% |
| Acessibilidade e qualidade do código | 10% |
| Testes e explicação do trabalho | 10% |

**Entrega:** pasta do website comprimida em ZIP e um pequeno README com o tema, instruções para abrir, funcionalidades, fontes das imagens e limitações conhecidas. Não incluas dados pessoais reais nos exemplos.

**Apresentação:** demonstra uma página num ecrã largo e num estreito, explica um seletor CSS e percorre a função JavaScript que criaste.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| Planear | Conteúdo antes do estilo | Esboço das três páginas |
| Testar | Verificar o comportamento | Lista de testes |
| Entregar | Projeto reproduzível | ZIP + README |

# Conferir sem deixar de pensar

## Desafios ao longo dos capítulos

**Tags mal encaixadas:** Olá turma! . Fecha strong antes de fechar o parágrafo que o contém.

**Expoente:** 2<sup>3</sup> apresenta 2³. HTML marca conteúdo, não calcula 8.

**Ligações e imagens:** os atributos em falta são href e alt. Em paginas/sobre.html, usa ../imagens/escola.jpg porque precisas de subir uma pasta.

**CSS sem separação:** p { color: blue; font-size: 18px; }. O ; separa declarações; não se coloca depois do seletor.

**Seletor descendente:** nav a não afeta uma ligação no footer que esteja fora de nav. O espaço indica descendência.

**Valores JavaScript:** 4 + 2 resulta em 6; "4" + 2 resulta em "42"; 4 === "4" é false, porque número e string têm tipos diferentes.

## Desafios A a D

**A:** rel="stylesheet" e src="js/script.js". Coloca link no head e, na estratégia usada nesta sebenta, script no fim de body.

**B:** .cartao { border: 1px solid teal; }. O ponto seleciona uma classe; # seleciona um id.

**C:** ficam dois cartões na segunda linha. As colunas definidas são três e a colocação automática preenche a primeira linha antes da seguinte.

**D:** getElementById recebe "mensagem", sem #; a propriedade é textContent, com C maiúsculo.

```
const texto = document.getElementById("mensagem");
if (texto) {
    texto.textContent = "Olá";
}
```

**AUTOAVALIAÇÃO**
Se a tua solução é diferente, verifica se cumpre os requisitos. O importante é conseguires explicar por que funciona. Volta ao exemplo original quando só consegues reproduzir código sem o compreender.

# Uma solução para o contador

## Desafio E: HTML

```
<button id="contar" type="button">Contar clique</button>
<p id="total" role="status">Cliques: 0</p>
```

## Desafio E: JavaScript externo

```
let cliques = 0;
const contar = document.querySelector("#contar");
const total = document.querySelector("#total");

if (contar && total) {
    contar.addEventListener("click", function () {
        cliques = cliques + 1;
        total.textContent = "Cliques: " + cliques;
    });
}
```

**Como funciona:** let permite atualizar o total. O código dentro da função é executado em cada click. textContent coloca texto no parágrafo. O teste if evita associar um evento a um elemento inexistente.

**Resultado:** começa em zero; depois de três cliques mostra “Cliques: 3”. Ao recarregar a página, volta a zero, porque o valor não é guardado de forma persistente.

**EXPERIMENTA TU**
Acrescenta um botão Repor. Na sua função, atribui 0 a cliques e atualiza o parágrafo. Testa a sequência: três cliques, repor, um clique. Resultado final esperado: 1.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| Estado | Valor que muda | let cliques = 0; |
| Evento | Desencadeia a função | addEventListener("click", …) |
| Atualização | Muda o valor e a página | cliques + 1 / textContent |
| Persistência | Não existe neste exemplo | Recarregar volta a zero |

# Ficheiros de apoio e referências

## Como utilizar os exemplos fornecidos

**1.** Extrai Exemplos_Clube_Web.zip.
**2.** Abre 01-html, 02-css ou 03-final com File > Open Folder no VS Code.
**3.** Abre index.html com Live Server.
**4.** Compara ficheiros das três versões e modifica uma cópia.
**5.** Consulta README.md para as funcionalidades e os testes sugeridos.

As ilustrações SVG incluídas foram criadas para este material e são locais. Os exemplos de áudio, vídeo e iframe são atividades separadas; exigem os ficheiros indicados ou acesso ao serviço externo. O formulário final não transmite nem guarda dados.

## Referências para aprofundar

HTML: estrutura, texto, ligações e imagens

CSS: regras, seletores e apresentação

JavaScript: aprendizagem e interação

Formulários: estrutura e funcionamento

iframe: atributos e restrições

Live Server: instalação e comandos

Nu HTML Checker: validação de HTML

Referências consultadas em 21 de setembro de 2026. Os exemplos e as atividades foram redigidos para esta sebenta; as ligações servem para consulta e aprofundamento.

**PARA O PROFESSOR**
Pode organizar cada aula em demonstração curta, prática acompanhada e verificação autónoma. Use as previsões dos alunos para identificar dificuldades; avalie também o processo de teste e a capacidade de explicar decisões.

## Ficha rápida de consulta

| Elemento / conceito | Função | Exemplo |
|---|---|---|
| HTML | Conteúdo | O que significa? |
| CSS | Apresentação | Como aparece e se adapta? |
| JavaScript | Comportamento | O que acontece ao interagir? |